

IMPLEMENTATION PROGRESS REPORT

Hostel Buddy

Smart Hostel Complaint Management System

30% · 50% · 80%

Group 19

Department of Computer Science and Engineering

IIIT Kottayam

Reporting period: 21 July — 5 September 2026

Status: 80% implementation complete

1. Team Members

#	Member Name	Roll Number
1	Pranjal Tyagi	2024BCD0053
2	Dinesh Saini	2024BCD0033
3	Ajinkya	2024BCS0173
4	Saurabh	2024BCS0185
5	Vijay	2024BCS0249

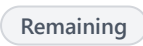
2. Purpose of This Report

This report records how **Hostel Buddy** was built from an empty repository to a working system that is **80% complete**. It is organised around the three implementation milestones set for the project — **30%, 50%** and **80%** — and for each one it answers the same three questions:

1. **What was required** at that milestone, and what we actually delivered.
2. **What went wrong** — the problems, bugs and design dilemmas we hit.
3. **How we solved them**, and what we learned that changed how we worked afterwards.

We have deliberately not written a report in which everything went smoothly. The most useful part of this document is Section 9, where every defect we found is listed with its root cause. Several of them were only found because we attacked our own running application rather than reading the code, and that is the single biggest lesson of the project so far.

2.1 How to read this document

Marker	Meaning
	Built, tested and committed.
	Part of the final 20%, listed in Section 11.
	A defect that reached working code and had to be fixed.
	An obstacle or a design dilemma — not a defect, but it cost us time.
	What we carried forward into the next milestone.

2.2 Development model

We worked in an **incremental, phase-wise model**. The project was cut into numbered phases before any code was written, each ending in a runnable system with its own tests. Nothing was merged until the whole suite passed. This is why a milestone can be demonstrated at any point: there is no phase in the history where the application does not start.

WHY NOT WATERFALL

Our requirements changed mid-project — the instructor supplied a revised ER model with three user roles after the two-role system was already working. A waterfall plan would have made that a crisis. Because every layer was isolated and covered by tests, the migration was carried out as seven small phases with the suite green at the end of each one.

3. Milestone Overview

80% complete — 30%, 50% and 80% milestones delivered; final 20% listed in Section 11

Milestone	Required deliverable	What we delivered	Period	Status
30%	Git repository, project skeleton, login module, database setup	Version-controlled repository, layered Express skeleton, six-table SQLite schema applied on boot, and a complete authentication module (registration, login, JWT, role guards)	21 July 2026	Done
50%	Major functional modules working end to end	Complaint module, staff complaint management with search and the status workflow, dashboards with analytics, and the full web front end over all of it	21 July – 24 Aug 2026	Done
80%	Begin integration testing; prepare unit test cases	Self-contained integration harness running 265 checks across six suites on every push, a full security and robustness audit, the three-role multi-hostel migration to the instructor's ER model, and video attachments	30 Aug – 5 Sept 2026	Done

3.1 The system as it stands today

6 database tables	3 user roles	17 API endpoints	12 web pages	265 automated checks	0 failing checks
-----------------------------	------------------------	----------------------------	------------------------	--------------------------------	----------------------------

3.2 Technology used

Layer	Technology	Why
Front end	HTML5, CSS3, vanilla JavaScript, Chart.js	No build step, so any team member can open the project and run it
Back end	Node.js + Express, layered as routes → controllers → services → repositories	Each layer has one job, which is what made the later migration survivable
Database	Node's built-in <code>node:sqlite</code>	No native compiler needed — see Problem P-1
Authentication	JWT + bcrypt, role-based access control	Stateless, matches the scaling requirement in the SRS
Security	Helmet headers with CSP, rate-limited login, byte-level upload checks	Added in the hardening phase and strengthened by the audit
Testing	Custom integration harness on Node's <code>fetch</code> , GitHub Actions	Runs the real HTTP API, not mocks

30% Foundation — Repository · Skeleton · Database · Login

Phases 0 – 1 · completed 21 July 2026

4.1 What this milestone required

A version-controlled project that starts, connects to a database, and can tell one user from another. Nothing user-facing yet — this milestone is about making sure the ground is solid before anything is built on it.

4.2 What we delivered

Deliverable	Detail
Git repository	Initialised with <code>.gitignore</code> (secrets, database files and uploads excluded), <code>.env.example</code> committed in place of the real <code>.env</code> , and one commit per phase so the history reads as a build log rather than a dump.
Project skeleton	Four-layer structure: <code>config/</code> (environment, constants), <code>db/</code> (connection, schema, seed), <code>middleware/</code> (auth, errors, logging, uploads, security) and <code>modules/</code> , where each feature owns its routes, controller, service and repository.
Database setup	<code>schema.sql</code> applied automatically at start-up and written to be idempotent, so a fresh clone needs no migration step. Enum columns carry SQL <code>CHECK</code> constraints; foreign keys are enforced with <code>PRAGMA foreign_keys = ON</code> .
Login module	Registration and login with bcrypt password hashing, JWT issue and verification, <code>requireAuth</code> and <code>requireRole</code> middleware, a seeded administrator account, and profile endpoints.
Health endpoint	<code>GET /api/health</code> , later used by the test harness to know when the server is ready.

4.3 Problems faced at this milestone, and how we solved them

PROBLEM P-1 — The database library would not install.

What happened: Our technical design named `better-sqlite3`. Installing it starts a native C++ build through `node-gyp`, which failed on a team machine that did not have Visual Studio build tools. A group project graded on several different machines cannot depend on a compiler toolchain being present.

How we solved it: Switched to Node's built-in `node:sqlite` module, which offers the same synchronous prepared-statement API with no native compilation. The SQL and the schema did not change at all — only the connection module.

Trade-off we accepted: `node:sqlite` is still marked experimental and prints a warning, which we silence in the npm scripts. Because all SQL is confined to the repository layer, moving back to `better-sqlite3` or on to PostgreSQL later would touch two files.

BUG B-2 — The login page told attackers which emails exist.

What happened: Signing in with an unknown address returned "no such user", while a known address with the wrong password returned "wrong password". Anyone could therefore discover which email addresses had accounts simply by trying them — a user-enumeration weakness.

How we solved it: Both failures now return the same generic `401 Invalid credentials`.

Lesson: Security messages should be deliberately vague; ordinary usability messages should be specific. Knowing which of the two you are writing is the whole skill.

PROBLEM P-3 — Where should business rules live?

The dilemma: The rule "a student may only edit a complaint while it is Pending" could be checked in the route handler in two lines. We debated this properly because it sets the pattern for the entire codebase.

What we decided: Enforce it in the **service layer**. A route-level check protects one route; if a second entry point ever calls the same operation — a bulk action, an admin tool, a test helper — a route-only guard is silently bypassed.

Why it mattered later: This decision is the reason the three-role migration at 80% was possible. Because every rule already lived in the services, changing who may call what meant changing the services, not hunting through routes.

PROBLEM P-4 — The seeded admin account silently failed to appear.

What happened: `.env` is deliberately not committed. On a fresh clone `ADMIN_PASSWORD` was therefore unset, the seed quietly created nothing, and the next person could not log in — with no error to explain why.

How we solved it: Added a start-up check that prints a clear warning when a required variable is missing, shipped `.env.example` as a template, and made the application **refuse to start in production** on default credentials or a default JWT secret.

4.4 Verification at 30%

Integration suite `tests/auth.test.js` — **24 checks**, all passing: registration validation, duplicate-email rejection, password hashing (never returned to the client), login success and failure, token verification, expiry handling, and the role guard refusing a student on an admin route.

50% Major Modules Functional

Phases 2 – 5 · 21 July – 24 August 2026

5.1 What this milestone required

The application should now do the job it exists for: a student raises a complaint and tracks it; an administrator works through the queue; both see meaningful statistics. Every feature must be reachable from a real interface, not only from the API.

5.2 What we delivered

Module	Capability
Complaint core	Create (with optional photo), list own complaints, view one, edit and delete — the last two allowed only while the complaint is Pending, and only for its author.
Staff complaint management	The whole queue with search by complaint id, student name or room; filters by category and status; pagination; and a status workflow with remarks.
Dashboards	A student dashboard counting their own complaints by state, and a staff dashboard with totals, a status breakdown, a category distribution chart and recent activity. All aggregation is done in SQL with <code>GROUP BY</code> rather than in JavaScript.
Web front end	Twelve pages built to the SRS wireframes — landing, login, register, both dashboards, raise a complaint, my complaints, profile, the staff queue, the student list, and a styled 404 — sharing one design system and one network layer.
Seed data	A repeatable demo dataset so any team member can show the product in a known state.

5.3 Problems faced at this milestone, and how we solved them

BUG B-1 — Photo uploads broke every complaint submission.

What happened: The create-complaint form sends `multipart/form-data` because it carries an image, but our JSON body-parser ran first and saw nothing. `req.body` came back empty, so validation rejected every single submission — including perfectly valid ones.

Root cause: Middleware order. The file-upload middleware must run *before* the fields are readable, and we had ordered the validator ahead of it.

How we solved it: Fixed the order to `upload → validate → controller`, and wrote the required order into the architecture document so nobody re-orders it by accident.

Lesson: Middleware order is part of the contract, not an implementation detail.

BUG B-3 — The resolution date kept moving.

What happened: We set `resolved_at = now` whenever the status was Resolved. So every later edit — even just correcting a typo in the remarks — pushed the resolution time forward, and moving a complaint from Resolved to Closed reset it again. The date became meaningless.

How we solved it: Set `resolved_at` only when it is currently empty and the complaint has reached Resolved.

Lesson: "The first time X happens" is a different rule from "whenever X is true", and they look identical until you read them carefully.

BUG B-4 — The client could award itself a status.

What happened: The create endpoint accepted a `status` field from the request body. A student could therefore submit a complaint already marked *Resolved* and skip the workflow entirely.

How we solved it: The service now **ignores** any status the client sends on create and forces `Pending`. Status changes only through the staff-only endpoint.

Lesson: Never accept a field from the client that the workflow owns.

BUG B-5 — Front-end pages each handled the token differently.

What happened: Pages made their own `fetch` calls. Some forgot the `Authorization` header, some forgot the content type, and an expired session produced a blank white page instead of sending the user to log in.

How we solved it: Wrote a single network layer, `js/api.js`, that attaches the token, sets headers, parses the response and redirects to login on a 401. Every page goes through it and no page does its own fetching.

Lesson: Centralise the network boundary. Cross-cutting logic repeated per page will be wrong on at least one page.

PROBLEM P-5 — Should uploaded photos be access-controlled?

The dilemma: Complaint photos are served as static files. That path is not behind the ownership check that protects the complaint record, so anyone holding the URL can open the image.

What we decided: Accept it for this version and **write it down** rather than leave it undiscovered. File names are a timestamp plus random hex, so they are effectively unguessable, and the URLs only ever appear inside access-controlled pages. Gating them properly means streaming every image through an authenticated route or issuing signed storage URLs.

Lesson: An accepted weakness that is documented is a decision. The same weakness undocumented is an oversight — and a reviewer cannot tell them apart unless you write it down.

PROBLEM P-6 — A strict security policy broke the landing page.

What happened: Adding a Content-Security-Policy of `script-src 'self'` blocked the one inline `<script>` we had — the "skip the landing page if already signed in" check. The choice looked like: weaken the policy, or lose the feature.

How we solved it: Neither. We moved the inline script into an external file, so the strict policy stayed exactly as it was.

Lesson: Keep scripts in external files from the very start. It costs nothing while you are writing them and it is what lets you ship a strict policy later.

5.4 Verification at 50%

Four integration suites — auth, complaints, admin and dashboard — totalling **103 checks**, all passing, plus a manual walkthrough of both user journeys in the browser with no console errors.

LESSON Reaching 50% taught us that a feature is not finished when the endpoint works. Three of the four bugs above were only visible from the front end, and one of them (B-1) made the flagship feature completely unusable while every server-side test was green.

80% Integration Testing · Audit · Three-Role Migration

Phase 6 + audit + Phases A – G · 30 August – 5 September 2026

6.1 What this milestone required

Begin integration testing and prepare test cases. We treated this as the milestone where the project stops being *written* and starts being *examined* — and it is where we found the most, by a wide margin.

6.2 What we delivered

1. **A self-contained integration harness** — one command runs everything.
2. **A full audit** of the running application against hostile input, which found fifteen defects that the test suite had not.
3. **The three-role, multi-hostel migration** to the ER model supplied by the instructor.
4. **Video attachments** on complaints, with content verified by file signature.
5. **Continuous integration** — the suite runs on GitHub Actions on every push.

6.3 Integration testing — how it works

A single command, `npm test`, starts the server on its own port against a **throwaway database and upload directory**, waits for the health endpoint to answer, seeds the hostels and accounts the scoping tests need, runs all six suites in order, then shuts the server down and deletes both. Development data is never touched.

Suite	Checks	What it proves
auth	47	Registration across three roles, login, JWT guards, profile rules
hostels	46	Public hostel list, super-admin-only hostel CRUD, manager provisioning
complaints	56	Student lifecycle, ownership, Pending-only edits, photo and video attachments
admin	52	Staff queue, search, filters, pagination, status transitions, hostel scoping
dashboard	23	Student and staff statistics, scoped aggregation
regression	41	Every audit defect, pinned so it cannot come back
Total 265 checks		All passing, in any order

6.4 The audit — how we tested

After the hardening phase we audited the whole application **against a running server**: the full suite, then manual probing of the API with malformed and hostile input, then a browser walkthrough of every page at desktop and mobile widths. **237 probe requests produced zero server errors**, and SQL injection, cross-site scripting, JWT forgery, mass assignment, ownership isolation and path traversal all held.

Fifteen things did break. The five most serious are described below; the complete list is in Section 9. Every one is now pinned by a check in the regression suite.

BUG B-8 — A JSON object was stored as the text "[object Object]".

What happened: Registering with `{"name": {"a": 1}}` returned success and created an account literally named `[object Object]`. The same worked for a complaint description and for admin remarks, so the junk was visible on the staff complaint table.

Root cause: Every validator converted first and checked second — `if (!name || !String(name).trim()).String({})` is a non-empty string, so an object sailed through. The helper that would have caught this, `isNonEmptyString`, existed in our validators file and **was exported but never called anywhere**.

How we solved it: Rewrote validation around explicit type predicates and used them at every entry point. A value of the wrong type is now rejected, never converted.

Lesson: Dead code in a validation module is a warning sign, not clutter. Check the type *before* you touch the value.

BUG B-9 — Complaints could be marked Pending while carrying a resolution date.

What happened: The status endpoint checked only that the new status was one of the four valid names — not that the move was legal. Closed could go back to Pending. Since a resolution date is deliberately never cleared, rolling a Resolved complaint backwards produced a row that was *Pending with a resolution date*, and the student could then edit a complaint that had already been resolved.

Root cause: The lifecycle was drawn in our diagrams and documented in the README, but only the *vocabulary* was enforced in code, never the *ordering*. Validating set membership felt like validating the rule.

How we solved it: Added a transition guard: a complaint's position in the lifecycle may stay the same (a remarks-only edit) or move forward, never backward; a backwards move returns `409 INVALID_TRANSITION`. The staff dropdown now offers only the statuses the server will accept, so the rule is visible rather than a surprise error.

Lesson: A state machine drawn in a diagram is not a state machine in the code. If order matters, something has to compare positions.

BUG B-12 — Upload checking trusted the browser's word.

What happened: We checked the file type using the MIME type in the upload header — which is simply whatever the client writes there. A text file declared as `image/png` was accepted, stored with a `.png` extension, and served back as an image.

How we solved it: After the file is saved we read its **first bytes and identify the real format from its signature** — PNG's eight-byte magic number, JPEG's `FF D8 FF`, `RIFF....WEBP`. Anything that is not genuinely one of those, or whose bytes disagree with its declared type, is deleted and rejected.

Lesson: Anything the client sends is a claim, not a fact. If a decision depends on what a file *is*, read the file.

BUG B-10 — Our own tests were polluting the demo database.

What happened: `npm test` assumed a server was already running and crashed with a raw connection error when it was not. Worse, the suites hit the development server, so they wrote into the real database and never cleaned up. After a few runs the staff screen was full of "Test Student", "Dash Tester" and the `[object Object]` account from B-8. One test even opened the database file directly by hardcoded path, reaching around the API entirely.

How we solved it: Built `tests/run.js` — the harness described in §6.3. The test that poked the database now advances the complaint through the staff endpoint like a real user.

Lesson: A test that needs a human to set something up first will eventually be run without it. And an integration test that reaches around the API is testing the database, not the application.

BUG B-16 — Brute-force protection was switched off in the only environment we ever run.

What happened: The login rate limiter was replaced by a do-nothing function unless the environment was `production` — and the project is demonstrated in `development`. The protection we had designed, documented and presented in our architecture review **had never actually run once**.

How we solved it: The limiter is now active in development and production alike and disabled only under the test runner. It counts *failed* attempts, so a real user signing in repeatedly is never locked out while a password guesser still is.

Lesson: Security that only exists in an environment you never run is not security. Disable it for tests, not for yourselves.

THE MAIN LESSON OF THIS MILESTONE Every one of the fifteen audit defects was found by **running** the application against hostile input — not by reading the code, and not by the test suite. The suites were green the entire time. They tested the paths we designed, which is exactly the input an audit must not limit itself to.

7. The Three-Role Migration

Part-way through this milestone the instructor supplied a revised entity-relationship model. It replaced our two-table, two-role design (*users* and *complaints*; student and admin) with **six tables and three roles**, adding hostels as first-class entities and specialising `USER` into `STUDENT`, `MANAGER` and `SUPER_ADMIN`.

PROBLEM P-7 — A change of data model with a working system on top of it.

The risk: This touched the schema, the seed, authentication, every complaint query, every dashboard figure, all guards and most of the front end. Done as one large change it would have left us with a broken application and no way to tell which part broke it.

How we handled it: Split into **seven phases (A – G)**, each one a separate commit with the full suite green before the next began. Phase A produced only a design document and a verified schema — no application code at all — so the model could be checked against the instructor's diagram before anything depended on it.

Result: The migration was completed without the application being broken at any commit.

7.1 Design decisions in the new model

- **Shared primary key.** Each subtype's `user_id` is both its primary key and its foreign key to `USER`, so one person cannot hold two identities.
- **A complaint references `STUDENT`, not `USER`.** The database itself refuses a complaint raised by a manager — the rule does not depend on our code being correct.
- **A complaint stores its own `hostel_id`.** It records the hostel the complaint was raised *against*, at that time, so history stays truthful if a student later changes hostel — and scoping becomes one indexed column instead of a join.
- **`SUPER_ADMIN` deliberately has no hostel.** Being unscoped is the point of the role, and the schema says so.

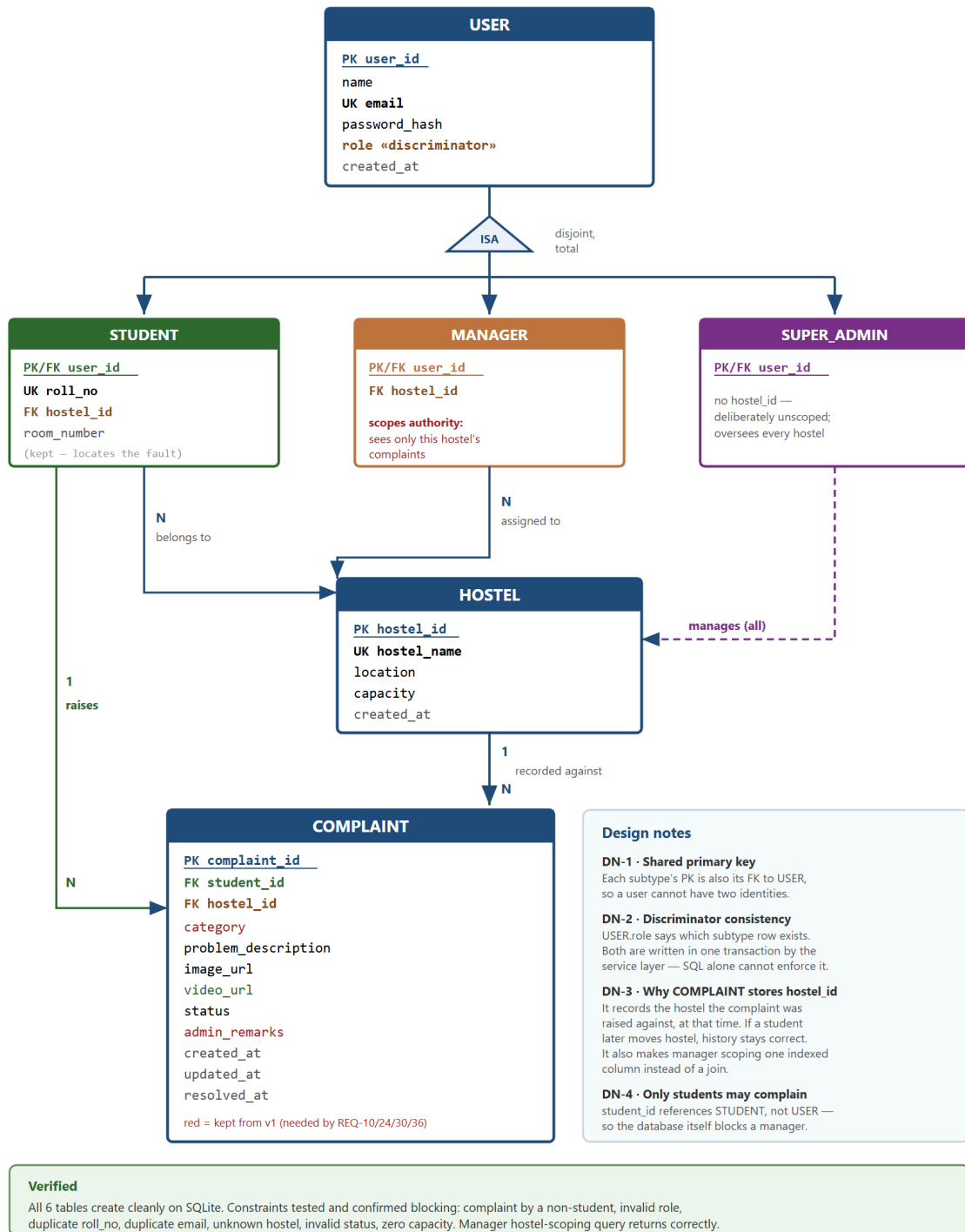
7.2 Hostel scoping — the new authorisation rule

A manager may only see and act on complaints belonging to their own hostel. This is a real authorisation boundary, so it is enforced in one place that every staff read and write passes through, and the manager's hostel is looked up **from their own database row on each request** — never taken from the request or the token, so it cannot be widened by asking. The filter is applied in SQL, so rows a manager may not see are never loaded at all.

TESTED BOTH WAYS The suite proves a manager cannot read *or* modify another hostel's complaint (both return 403), that the complaint is verifiably untouched afterwards, and that searching for another hostel's complaint by its id returns nothing.

Hostel Buddy — ER Schema (v2)

USER specialised into STUDENT / MANAGER / SUPER_ADMIN · HOSTEL 1—N COMPLAINT



Designed in Phase A, applied in Phase B — this is the schema the running application uses

Figure 1 — The implemented entity-relationship model: USER specialised into STUDENT, MANAGER and SUPER_ADMIN, with HOSTEL 1—N COMPLAINT.

8. The Product at 80%

Role	What they can do
Student	Register with a roll number and their hostel; raise a complaint with an optional photo and video ; track it through Pending → In Progress → Resolved → Closed; read the manager's remarks; edit or delete it while it is still Pending; see their own statistics.
Hostel manager	The complaint queue for their own hostel — search, filter, paginate — with the student list and every dashboard figure narrowed the same way. Advances status and leaves remarks. Provisioned by the super admin; there is no public sign-up for this role.
Super admin	The same views unscoped, across every hostel; creates, renames and removes hostels; creates manager accounts and binds each to a hostel.

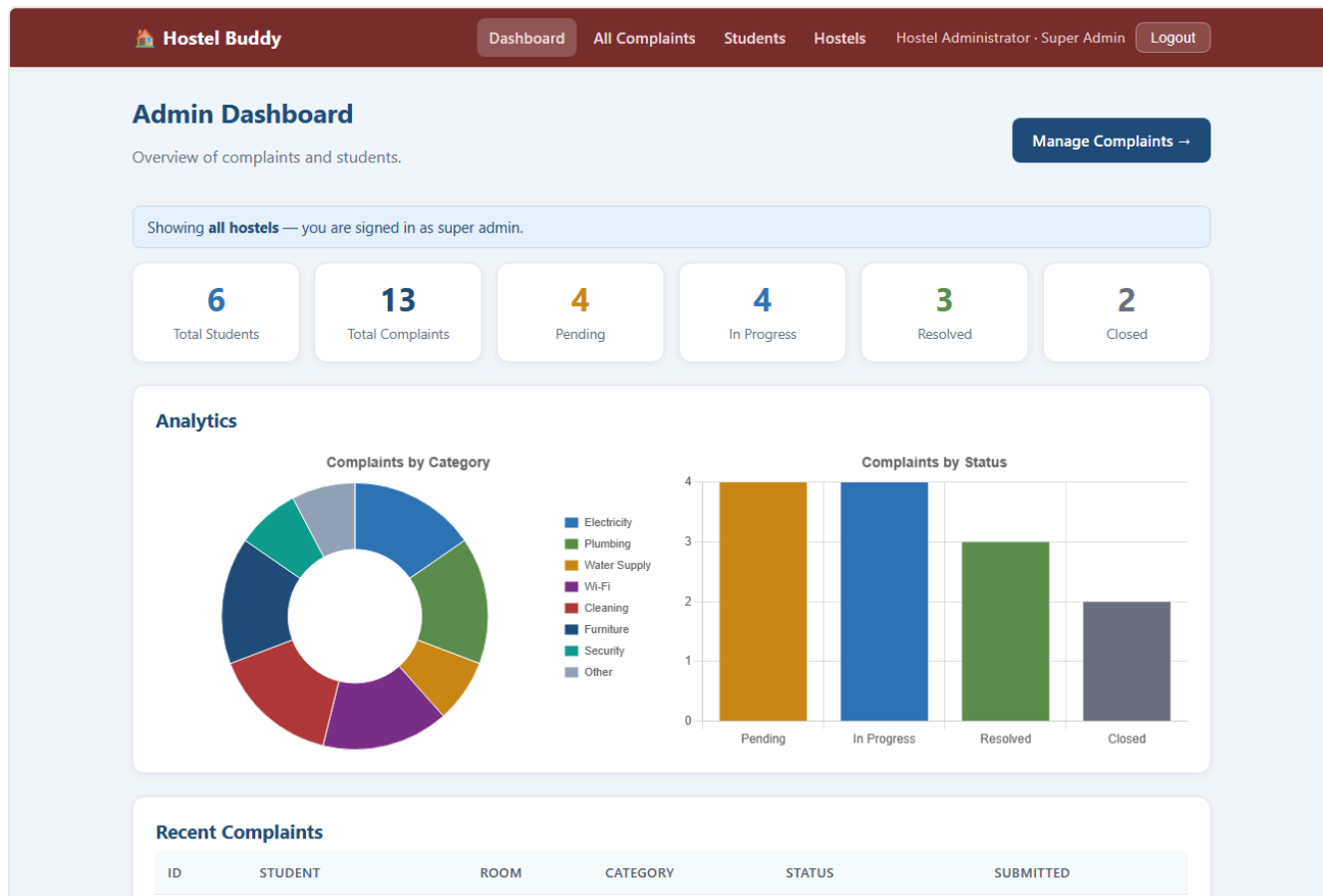


Figure 2 — Staff dashboard: totals, status breakdown and category distribution. For a manager every figure on this page is limited to their own hostel.

Hostel Buddy

Dashboard
All Complaints
Students
Hostels
Hostel Administrator · Super Admin
Logout

All Complaints

Search

Complaint ID, student name, or room number...

Category

All

Status

All

Search

Clear

ID	STUDENT	HOSTEL	ROOM	CATEGORY	DESCRIPTION	STATUS	SUBMITTED	ACTION
#8	Priya Singh	Ramanujan Hostel	B-210	Wi-Fi	Wi-Fi very slow in the evening.	Pending	Aug 31, 2026	Manage
#1	Rahul Sharma	Aryabhata Hostel	B-204	Wi-Fi	No Wi-Fi signal in room B-204 since morning.	Pending	Aug 31, 2026	Manage
#10	Amit Kumar	Bhaskara Hostel	D-305	Cleaning	Washroom needs deep cleaning.	Pending	Aug 30, 2026	Manage
#4	Ananya Verma	Aryabhata Hostel	A-112	Plumbing	Leaking tap in the washroom on A-wing.	Pending	Aug 30, 2026	Manage
#6	Karan Nair	Ramanujan Hostel	C-007	Security	Main gate CCTV appears to be off.	In Progress	Aug 29, 2026	Manage
#2	Rahul Sharma	Aryabhata Hostel	B-204	Electricity	Ceiling fan not working.	In Progress	Aug 28, 2026	Manage
#11	Amit Kumar	Bhaskara Hostel	D-305	Plumbing	Flush not working in D-305.	In Progress	Aug 27, 2026	Manage
#5	Ananya Verma	Aryabhata Hostel	A-112	Water Supply	No hot water in the mornings.	In Progress	Aug 26, 2026	Manage
#12	Sneha Rao	Bhaskara Hostel	D-110	Furniture	Cupboard door hinge is loose.	Resolved	Aug 22, 2026	Manage

Figure 3 — The complaint queue with search, filters and pagination.

Hostel Buddy

Dashboard
Raise Complaint
My Complaints
Profile
Rahul Sharma
Logout

Raise a Complaint

Category

Select a category...

Description

Describe the issue in detail...

0/1000 characters

Attach an image (optional)

Choose File No file chosen

PNG, JPEG or WEBP, up to 5 MB.

Attach a video (optional)

Choose File No file chosen

MP4 or WEBM, up to 30 MB. Useful for faults you can only show in motion — a flickering light or a dripping pipe.

Submit Complaint

Cancel

Figure 4 — Raising a complaint, with optional photo and video attachments.

8.1 Video attachments

Some faults only make sense in motion — a light that flickers at a particular rate, a tap that drips every few seconds. A still photo cannot show either, so a complaint may now carry one image **and** one video (MP4 or WebM, up to 30 MB against the image's 5 MB).

The video is identified by its leading bytes, as images already were, and the check is deliberately narrow: only formats a browser can actually play are stored, because storing a file the complaint page cannot show is worse than refusing it.

9. Every Defect at a Glance

All twenty-three problems and defects encountered so far, with where they were found and how they were resolved. Entries marked **R** are pinned by a check in the regression suite.

Ref	Problem	Root cause and fix	Found at	R
P-1	Database library needed a C++ compiler	Native build failed without Visual Studio; switched to Node's built-in SQLite	30%	—
B-2	Login revealed which emails exist	Two different error messages; now one generic 401	30%	—
P-3	Where to enforce business rules	Decided on the service layer, not routes — what made the later migration possible	30%	—
P-4	Seeded admin silently missing	Unset environment variable; added start-up warning and production refusal	30%	—
B-1	Every complaint submission failed with a photo	Middleware ordered validator before upload; order corrected and documented	50%	—
B-3	Resolution date kept moving	Unconditional assignment; now set once, on first reaching Resolved	50%	R
B-4	Client could set its own status	Accepted a client field the workflow owns; now forced to Pending	50%	R
B-5	Inconsistent tokens and blank pages on expiry	Ad-hoc fetches per page; centralised into one network layer	50%	—
P-5	Uploaded photos not access-controlled	Accepted for this version with unguessable names; documented, not hidden	50%	—
P-6	Security policy blocked the landing page	Moved the inline script to a file instead of weakening the policy	50%	—
B-6	Tests passed alone, failed together	Shared state and absolute counts; every suite now owns its data and asserts deltas	80%	R
B-8	Object stored as "[object Object]"	Validators coerced before checking; rewritten around type predicates	80%	R
B-9	Status could move backwards	Only vocabulary enforced, not ordering; added a transition guard	80%	R
B-10	Tests wrote into the demo database	No isolation; built a harness with a throwaway database and server	80%	R
B-11	A requirement was built but unreachable	Endpoint existed with no page; added the student list screen and nav link	80%	R
B-12	Upload type trusted the browser	Declared MIME type treated as fact; now verified by file signature	80%	R
B-13	Framework errors leaked to users	Handler returned raw messages; only our own errors keep their text	80%	R
B-14	"%" in search matched everything	Parameterisation protects the parser, not LIKE's pattern language; escaped it	80%	R
B-15	Long passwords silently truncated	bcrypt reads 72 bytes; longer passwords now rejected with a clear message	80%	R
B-16	Rate limiting never actually ran	Enabled only in production, which we never run; now on everywhere but tests	80%	R
B-17	Six small correctness and layout defects	Page clamping, image removal, styled 404, dialog timer, mobile actions, logging	80%	R
P-7	Data model changed under a working system	Split into seven phases, suite green at each; design verified before any code	80%	—
B-18	Signed-out users still saw the dashboard	Guard ran after the page painted; moved into the head, before any markup	80%	—

9.1 Where our defects came from

How it was found	Entries	What that tells us
Auditing the running application with hostile input	11	By far the most productive technique. Counting B-17's six grouped items individually, the audit accounted for 15 separate defects — and the test suite was green before it started and green after it finished
Building the front end over a finished API	3	A working endpoint is not a working feature; one of these (B-1) made the flagship feature unusable while every server test passed
Ordinary development and code review	3	Caught early and cheaply, before anyone else depended on them
Design debates held before writing the code	6	Cheapest of all — these are decisions and trade-offs that never became defects
Total 23 entries		All resolved

10. Test Cases Prepared

A representative selection from the 265 automated checks. Each names its precondition, the action, and the result the server must produce; all of them run over real HTTP against a real database, not against mocks.

10.1 Authentication and accounts

ID	Test case	Input	Expected result
TC-A1	Register a valid student	Name, email, password, roll number, hostel	201 with a token; no password hash in the response
TC-A2	Reject a duplicate email	An email already registered	409 EMAIL_TAKEN
TC-A3	Reject a duplicate roll number	A roll number already registered	409 ROLL_NO_TAKEN
TC-A4	Reject a non-string name	<code>{"name": {"a": 1}}</code>	400 — never stored as text
TC-A5	Do not reveal unknown emails	Unknown email vs wrong password	Identical 401 for both
TC-A6	Reject a forged token	A token signed with the wrong secret	401 UNAUTHENTICATED
TC-A7	Self-registration cannot create staff	<code>role: "super_admin"</code> in the body	Account created as a student regardless
TC-A8	Reject an over-long password	A password above 72 bytes	400 with an explanatory message

10.2 Complaints and the lifecycle

ID	Test case	Input	Expected result
TC-C1	Create a complaint	Valid category and description	201, status forced to Pending
TC-C2	Attach a photo and a video	A real PNG and a real MP4	201; both stored and served
TC-C3	Reject a disguised file	A text file declared <code>image/png</code>	400 INVALID_FILE_TYPE ; file deleted
TC-C4	Reject an unplayable video	A QuickTime clip declared <code>video/mp4</code>	400 — brand not on the allowlist
TC-C5	Enforce the per-field size limit	A 6 MB image	400 FILE_TOO_LARGE
TC-C6	Owner-only access	Student B opens Student A's complaint	403 FORBIDDEN
TC-C7	Edit only while Pending	Edit a complaint already In Progress	409 NOT_PENDING
TC-C8	Status cannot move backwards	Closed → In Progress	409 INVALID_TRANSITION
TC-C9	Resolution date is set once	Resolved, then remarks edited	Date unchanged by the second edit
TC-C10	No open complaint has a resolution date	Query the whole table	Zero such rows
TC-C11	Deleting removes the attachments	Delete a complaint with both files	Row gone; both URLs return 404

10.3 Roles and hostel scoping

ID	Test case	Input	Expected result
TC-S1	Manager sees only their hostel	Manager lists all complaints	Only their own hostel's rows

TC-S2	Manager cannot read another hostel	Open a complaint from another hostel	403 FORBIDDEN
TC-S3	Manager cannot modify another hostel	Change its status	403; complaint verifiably untouched
TC-S4	Scope cannot be widened by searching	Search another hostel's complaint id	No results
TC-S5	Manager cannot create hostels	Manager posts a new hostel	403 — would let them redraw their own authority
TC-S6	Manager cannot provision managers	Manager creates a manager account	403
TC-S7	A manager must have a hostel	Provision one with no hostel	400 VALIDATION_ERROR
TC-S8	Students cannot reach staff routes	Student calls the queue and staff dashboard	403 on both
TC-S9	A hostel in use cannot be deleted	Delete a hostel with students	409 HOSTEL_IN_USE

10.4 Robustness and security

ID	Test case	Input	Expected result
TC-R1	Search wildcards are literal	Search for % and _	No match-everything behaviour
TC-R2	Page numbers are clamped	?page=99999	Clamped to the last real page
TC-R3	No framework internals leak	Malformed JSON body	400 with our own message and code
TC-R4	Unknown pages get our 404	A mistyped URL	The styled 404 page; JSON for API routes
TC-R5	Student list is staff-only	Student requests it	403; no password hashes ever returned

UNIT-LEVEL CHECKS Alongside these, the pure functions that carry the rules — the status-transition comparison, the LIKE escaper, the type predicates and the upload signature detector — are each exercised directly by the cases above, since every one of them was written in response to a specific defect.

11. The Remaining 20%

Item	What is left to do	Status
Session revocation	Logging out clears the browser's session but the token itself stays valid until it expires. A server-side invalidation step is designed and scheduled.	Remaining
Super-admin scope review	Deciding whether the super admin should act on complaints directly or oversee them read-only, leaving the queue to hostel managers.	Remaining
Document revision	The SRS, the SADD and the architecture presentation still describe the earlier two-role design and must be brought onto the three-role model — ER diagram, data dictionary, class and sequence diagrams, and the traceability matrix.	Remaining
Regression coverage	Pinning the two most recent fixes with their own checks, and folding the browser-level page-guard verification into the standard suite.	Remaining
Demonstration video	A recorded walkthrough of all three roles, end to end.	Remaining
Deployment	Hosting the application so it can be reached without a local checkout.	Remaining

12. Conclusion

At 80%, Hostel Buddy is a working three-role, multi-hostel complaint management system. Every functional requirement in the SRS is implemented and reachable from the interface, the entity model matches the one supplied by the instructor, and **265 automated checks run on every push and all of them pass.**

The part of this milestone we would defend most strongly is the audit. Our test suite was green before it began and green after it ended, yet the audit found fifteen defects — including one that let a complaint sit in a contradictory state, one that stored arbitrary objects as text, and one that meant the brute-force protection we had presented in our architecture review had never run at all. That gap between "the tests pass" and "the system is correct" is the most valuable thing the project has taught us, and it changed how we work: **we now try to break each feature before we call it finished.**

The remaining 20% is largely consolidation — finishing the session-revocation work, bringing the formal documents onto the current data model, and recording the demonstration. The engineering is in place.

Group 19 — Team Signature

Faculty Signature